Analyze my entire workspace and understand how UI is implemented in the frontend folder.
I want to connect the jira with the UI.
I want to fetch all the user stories and display it in the UI.
Create a "Fetch User stories" button above the form-container in the UI. The button should be yellow in color.
Generate the code with all the requirements and go step by step upon my approval

Reviewing the frontend implementation and related backend integration to understand how the app is wired.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#1-1), lines 1 to 260

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/api.ts)

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/types.ts)

Read [](file:///d%3A/SAMPLE/sample/samples/backend/src/routes/generate.ts)

Read [](file:///d%3A/SAMPLE/sample/samples/backend/src/server.ts)

Checking the frontend wiring and any existing API support so I can map the Jira integration cleanly.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/package.json)

Read [](file:///d%3A/SAMPLE/sample/samples/backend/package.json)

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/main.tsx)

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/tsconfig.json)

I’ve confirmed the frontend state and backend API shape. Next I’m checking whether there’s any existing Jira or user-story endpoint support in the workspace.

Searched for regex `jira|Jira|user story|user-story|stories|story` (`**/*.{ts,tsx,js,json,md}`), 25 results

I’m verifying the backend and workspace structure for available routes and whether Jira data fetching is already supported.

Read [](file:///d%3A/SAMPLE/sample/samples/backend/src)

I’ve confirmed there is no existing Jira or story-fetching API in the workspace. Next I’ll inspect the full backend route structure and root configuration so I can map the required changes precisely.

Searched for regex `use\('/api|app.use\('/api|router|express.Router|fetch\(` (`**/*.{ts,tsx,js,json}`), 8 results

Read [](file:///d%3A/SAMPLE/sample/samples/package.json)

I'm checking whether there is existing environment or Jira-related config to support a backend Jira integration.

Searched for files matching `**/.env*`, no matches

Current frontend UI implementation

The current UI is implemented entirely in App.tsx as a single React component:

- Uses `useState` for:
  - `formData` for the story title, acceptance criteria, description, additional info
  - `results` for generated test cases
  - `isLoading`, `error`, and `expandedTestCases`
- Submits form data via `generateTests()` from api.ts
- Displays results in a table with expandable rows for test steps
- Styles are inline inside the `App` component via a `<style>` block

api.ts currently supports only:

- `POST /api/generate-tests`

types.ts defines the existing request/response types for test generation.

Current backend state

The backend in server.ts currently only exposes:

- `GET /api/health`
- `POST /api/generate-tests`

There is no Jira integration or Jira story-fetching route in the current workspace.

What is needed to connect Jira and show stories in the UI

1. Add a backend Jira fetch endpoint
Because Jira credentials must be kept secure, the cleanest approach is:

- Add a backend route such as:
  - `GET /api/jira/stories`
- This route should:
  - read Jira config from env vars

- call Jira REST API
- return a simplified list of user stories

Suggested env variables:
- `JIRA_BASE_URL`
- `JIRA_EMAIL`
- `JIRA_API_TOKEN`
- `JIRA_PROJECT_KEY`

Example Jira REST call:
- `GET ${JIRA_BASE_URL}/rest/api/3/search?jql=project=${JIRA_PROJECT_KEY}+AND+issuetype=Story`

---

2. Add frontend API helper
Update api.ts to export something like:

- `fetchUserStories(): Promise<JiraStory[]>`

This will call:
- `${API_BASE_URL}/jira/stories`

---

3. Add Jira story types
Update types.ts with a new interface, for example:

- `JiraStory`
- `FetchStoriesResponse`

So the UI can strongly type the fetched story list.

---

4. Update App.tsx
Add state and behavior for Jira stories:

- `jiraStories`
- `isFetchingStories`
- `fetchStoriesError`

Add a yellow button above `.form-container`:

- `Fetch User stories`
- Yellow background
- Calls `handleFetchStories`

Add a UI section to display fetched stories, such as:

- a list or table of issue key / summary / status
- optionally let the user click one to prefill the form

Proposed UI placement

In App.tsx, insert the button above the form block:

- before `<form ... className="form-container">`
- using a new CSS class like `.fetch-btn`

Example layout:
- Header
- `Fetch User stories` button
- Form container
- Error/loading/results sections

Example implementation plan

Step 1
Create backend Jira route and register it in server.ts.

Step 2
Update api.ts with `fetchUserStories()`.

Step 3
Update types.ts with Jira story types.

Step 4
Update App.tsx:
- add Jira story state
- add fetch handler
- add yellow button
- add display block for fetched stories

Step 5
Add environment configuration for Jira credentials.

Next step

I can now generate the exact code changes for:

- `backend/src/routes/jira.ts`
- server.ts
- api.ts
- types.ts
- App.tsx

Please confirm and I'll provide the code step by step.

Firstly, provide all the code changes needed in frontend folder - UI changes. Also, give me the difference between existing files and new updated files. Go step by step upon my approval

Frontend UI changes needed

I reviewed the frontend and the changes required inside src are:

1. types.ts
   - Add a Jira story type so the UI can strongly type the response.

2. api.ts
   - Add a `fetchUserStories()` helper that calls `GET /api/jira/stories`.
   - Keep the existing `generateTests()` method.

3. App.tsx
   - Add Jira story state and fetch state
   - Add a yellow `Fetch User stories` button above the form
   - Add a Jira story display section
   - Add supporting styles for the button and story cards

> Note: this frontend work assumes the backend exposes `GET /api/jira/stories`. The backend route still needs to be implemented separately for the button to work end-to-end.

File differences summary

types.ts
Existing:
- `GenerateRequest`
- `TestCase`
- `CostBreakdown`
- `GenerateResponse`

Updated:
- Add `JiraStory` interface

api.ts

Existing:
- `generateTests()`

Updated:
- Add `fetchUserStories()`
- Import `JiraStory` type

App.tsx
Existing:
- Form submit UI only
- No Jira story state
- No fetch button
- No Jira story results display

Updated:
- New imports
- New state hooks:
  - `jiraStories`
  - `isFetchingStories`
  - `fetchStoriesError`
- New handler:
  - `handleFetchStories`
- New button above `<form>`
- New section showing fetched stories
- New styles for `.fetch-btn`, `.stories-container`, `.story-card`, etc.

Exact code change outline

1) types.ts

Add:

```ts
export interface JiraStory {
  key: string
  summary: string
  status: string
  assignee?: string
  project?: string
}
```

---

2) api.ts

Update the imports and add:

```ts
import { GenerateRequest, GenerateResponse, JiraStory } from './types'

export async function fetchUserStories(): Promise<JiraStory[]> {
  try {
    const response = await fetch(`${API_BASE_URL}/jira/stories`)

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))
      throw new Error(errorData.error || `HTTP error! status: ${response.status}`)
    }

    const data: JiraStory[] = await response.json()
    return data
  } catch (error) {
    console.error('Error fetching Jira stories:', error)
    throw error instanceof Error ? error : new Error('Unknown error occurred')
  }
}
```

---

3) App.tsx

Key additions:

```tsx
import { useState } from 'react'
import { generateTests, fetchUserStories } from './api'
import { GenerateRequest, GenerateResponse, TestCase, JiraStory } from './types'
```

Add state:

```tsx
const [jiraStories, setJiraStories] = useState<JiraStory[]>([])
const [isFetchingStories, setIsFetchingStories] = useState<boolean>(false)
const [fetchStoriesError, setFetchStoriesError] = useState<string | null>(null)
```

Add handler:

```tsx
const handleFetchStories = async () => {
  setIsFetchingStories(true)
```

```
  setFetchStoriesError(null)

  try {
    const stories = await fetchUserStories()
    setJiraStories(stories)
  } catch (err) {
    setFetchStoriesError(err instanceof Error ? err.message : 'Failed to fetch Jira stories')
  } finally {
    setIsFetchingStories(false)
  }
}
```

Add button markup above the form:

```tsx
<div className="fetch-container">
  <button
    type="button"
    className="fetch-btn"
    onClick={handleFetchStories}
    disabled={isFetchingStories}
  >
    {isFetchingStories ? 'Fetching stories...' : 'Fetch User stories'}
  </button>
</div>
```

Add Jira story list section:

```tsx
{jiraStories.length > 0 && (
  <div className="stories-container">
    <h2>Fetched Jira Stories</h2>
    <div className="stories-list">
      {jiraStories.map((story) => (
        <div key={story.key} className="story-card">
          <div className="story-key">{story.key}</div>
          <div className="story-summary">{story.summary}</div>
          <div className="story-details">
            <span>{story.status}</span>
            {story.assignee && <span> • {story.assignee}</span>}
            {story.project && <span> • {story.project}</span>}
          </div>
        </div>
      ))}
    </div>
  </div>
```

)}
```

Add styles:

```css
.fetch-container {
  margin-bottom: 20px;
  text-align: center;
}

.fetch-btn {
  background: #f1c40f;
  color: #1f1f1f;
  border: none;
  padding: 12px 24px;
  border-radius: 6px;
  font-size: 16px;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 0.2s;
}

.fetch-btn:hover:not(:disabled) {
  background: #d4ac0d;
}

.fetch-btn:disabled {
  background: #f7dc6f;
  cursor: not-allowed;
}

.stories-container {
  background: white;
  border-radius: 8px;
  padding: 24px;
  margin-bottom: 30px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.08);
}

.stories-list {
  display: grid;
  gap: 16px;
  margin-top: 16px;
}

.story-card {
  border: 1px solid #e1e8ed;
```
```

```
  border-radius: 8px;
  padding: 18px;
  background: #fcfcfc;
}

.story-key {
  font-weight: 700;
  margin-bottom: 8px;
  color: #2c3e50;
}

.story-summary {
  color: #444;
  margin-bottom: 10px;
}

.story-details {
  color: #666;
  font-size: 14px;
}
```

Step-by-step proposal

1. Confirm the frontend changes above.
2. I can then give you the exact updated App.tsx, api.ts, and types.ts contents for replacement.
3. After that, we can implement backend Jira support so the new UI works end-to-end.

Please approve this plan and I'll provide the exact updated file code for the frontend.

Proceed

Frontend changes approved — step-by-step

I'm giving you the exact UI updates for the frontend only. After this, I can do the backend Jira route so the button works end-to-end.

Step 1 — types.ts

Change
Add a new Jira story type so the UI can type the fetched story data.

Updated file
```ts
```

```
export interface GenerateRequest {
  storyTitle: string
  acceptanceCriteria: string
  description?: string
  additionalInfo?: string
}

export interface TestCase {
  id: string
  title: string
  steps: string[]
  testData?: string
  expectedResult: string
  category: string
}

export interface CostBreakdown {
  inputCost: number
  outputCost: number
  totalCost: number
  currency: 'USD'
  estimated: boolean
}

export interface GenerateResponse {
  cases: TestCase[]
  model?: string
  promptTokens: number
  completionTokens: number
  cost?: CostBreakdown
}

export interface JiraStory {
  key: string
  summary: string
  status: string
  assignee?: string
  project?: string
}
```

Step 2 — api.ts

Change
Add a new `fetchUserStories()` API helper that calls `GET /api/jira/stories`.

Updated file
```ts
import { GenerateRequest, GenerateResponse, JiraStory } from './types'

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8081/api'

export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {
  try {
    const response = await fetch(`${API_BASE_URL}/generate-tests`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(request),
    })

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))
      throw new Error(errorData.error || `HTTP error! status: ${response.status}`)
    }

    const data: GenerateResponse = await response.json()
    return data
  } catch (error) {
    console.error('Error generating tests:', error)
    throw error instanceof Error ? error : new Error('Unknown error occurred')
  }
}

export async function fetchUserStories(): Promise<JiraStory[]> {
  try {
    const response = await fetch(`${API_BASE_URL}/jira/stories`)

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))
      throw new Error(errorData.error || `HTTP error! status: ${response.status}`)
    }

    const data: JiraStory[] = await response.json()
    return data
  } catch (error) {
    console.error('Error fetching Jira stories:', error)
    throw error instanceof Error ? error : new Error('Unknown error occurred')
  }
}
```

Step 3 — App.tsx

Change
Add:
- new imports
- Jira story state
- fetch handler
- yellow `Fetch User stories` button above the form
- Jira story display section
- styles for the new button and story cards

Updated file
```tsx
import { useState } from 'react'
import { generateTests, fetchUserStories } from './api'
import { GenerateRequest, GenerateResponse, TestCase, JiraStory } from './types'

function App() {
  const [formData, setFormData] = useState<GenerateRequest>({
    storyTitle: '',
    acceptanceCriteria: '',
    description: '',
    additionalInfo: ''
  })
  const [jiraStories, setJiraStories] = useState<JiraStory[]>([])
  const [isFetchingStories, setIsFetchingStories] = useState<boolean>(false)
  const [fetchStoriesError, setFetchStoriesError] = useState<string | null>(null)
  const [results, setResults] = useState<GenerateResponse | null>(null)
  const [isLoading, setIsLoading] = useState<boolean>(false)
  const [error, setError] = useState<string | null>(null)
  const [expandedTestCases, setExpandedTestCases] = useState<Set<string>>(new Set())

  const toggleTestCaseExpansion = (testCaseId: string) => {
    const newExpanded = new Set(expandedTestCases)
    if (newExpanded.has(testCaseId)) {
      newExpanded.delete(testCaseId)
    } else {
      newExpanded.add(testCaseId)
    }
    setExpandedTestCases(newExpanded)
  }

  const handleInputChange = (field: keyof GenerateRequest, value: string) => {
    setFormData(prev => ({ ...prev, [field]: value }))
  }

  const handleFetchStories = async () => {
```

```
  setIsFetchingStories(true)
  setFetchStoriesError(null)
  setError(null)

  try {
    const stories = await fetchUserStories()
    setJiraStories(stories)
  } catch (err) {
    setFetchStoriesError(err instanceof Error ? err.message : 'Failed to fetch Jira stories')
  } finally {
    setIsFetchingStories(false)
  }
}

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault()

  if (!formData.storyTitle.trim() || !formData.acceptanceCriteria.trim()) {
    setError('Story Title and Acceptance Criteria are required')
    return
  }

  setIsLoading(true)
  setError(null)

  try {
    const response = await generateTests(formData)
    setResults(response)
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Failed to generate tests')
  } finally {
    setIsLoading(false)
  }
}

return (
  <div>
    <style>{`
      * {
        box-sizing: border-box;
        margin: 0;
        padding: 0;
      }

      body {
        font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', sans-serif;
        background-color: #f5f5f5;
        color: #333;
```

```
  line-height: 1.6;
}

.container {
  max-width: 95%;
  width: 100%;
  margin: 0 auto;
  padding: 20px;
  min-height: 100vh;
}

@media (min-width: 768px) {
  .container {
    max-width: 90%;
    padding: 30px;
  }
}

@media (min-width: 1024px) {
  .container {
    max-width: 85%;
    padding: 40px;
  }
}

@media (min-width: 1440px) {
  .container {
    max-width: 1800px;
    padding: 50px;
  }
}

.header {
  text-align: center;
  margin-bottom: 40px;
}

.title {
  font-size: 2.5rem;
  color: #2c3e50;
  margin-bottom: 10px;
}

.subtitle {
  color: #666;
  font-size: 1.1rem;
}
```

```
.fetch-container {
  margin-bottom: 20px;
  text-align: center;
}

.fetch-btn {
  background: #f1c40f;
  color: #1f1f1f;
  border: none;
  padding: 12px 24px;
  border-radius: 6px;
  font-size: 16px;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 0.2s;
}

.fetch-btn:hover:not(:disabled) {
  background: #d4ac0d;
}

.fetch-btn:disabled {
  background: #f7dc6f;
  cursor: not-allowed;
}

.stories-container {
  background: white;
  border-radius: 8px;
  padding: 24px;
  margin-bottom: 30px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.08);
}

.stories-list {
  display: grid;
  gap: 16px;
  margin-top: 16px;
}

.story-card {
  border: 1px solid #e1e8ed;
  border-radius: 8px;
  padding: 18px;
  background: #fcfcfc;
}

.story-key {
```

```
  font-weight: 700;
  color: #2c3e50;
  margin-bottom: 8px;
}

.story-summary {
  color: #444;
  margin-bottom: 10px;
}

.story-details {
  color: #666;
  font-size: 14px;
}

.form-container {
  background: white;
  border-radius: 8px;
  padding: 30px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  margin-bottom: 30px;
}

.form-group {
  margin-bottom: 20px;
}

.form-label {
  display: block;
  font-weight: 600;
  margin-bottom: 8px;
  color: #2c3e50;
}

.form-input, .form-textarea {
  width: 100%;
  padding: 12px;
  border: 2px solid #e1e8ed;
  border-radius: 6px;
  font-size: 14px;
  transition: border-color 0.2s;
}

.form-input:focus, .form-textarea:focus {
  outline: none;
  border-color: #3498db;
}
```

```
.form-textarea {
  resize: vertical;
  min-height: 100px;
}

.submit-btn {
  background: #3498db;
  color: white;
  border: none;
  padding: 12px 24px;
  border-radius: 6px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}

.submit-btn:hover:not(:disabled) {
  background: #2980b9;
}

.submit-btn:disabled {
  background: #bdc3c7;
  cursor: not-allowed;
}

.error-banner {
  background: #e74c3c;
  color: white;
  padding: 15px;
  border-radius: 6px;
  margin-bottom: 20px;
}

.loading {
  text-align: center;
  padding: 40px;
  color: #666;
  font-size: 18px;
}

.results-container {
  background: white;
  border-radius: 8px;
  padding: 30px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
```

```
.results-header {
  margin-bottom: 20px;
  padding-bottom: 15px;
  border-bottom: 2px solid #e1e8ed;
}

.results-title {
  font-size: 1.8rem;
  color: #2c3e50;
  margin-bottom: 10px;
}

.results-meta {
  color: #666;
  font-size: 14px;
}

.table-container {
  overflow-x: auto;
}

.results-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}

.results-table th,
.results-table td {
  padding: 12px;
  text-align: left;
  border-bottom: 1px solid #e1e8ed;
}

.results-table th {
  background: #f8f9fa;
  font-weight: 600;
  color: #2c3e50;
}

.results-table tr:hover {
  background: #f8f9fa;
}

.category-positive { color: #27ae60; font-weight: 600; }
.category-negative { color: #e74c3c; font-weight: 600; }
.category-edge { color: #f39c12; font-weight: 600; }
.category-authorization { color: #9b59b6; font-weight: 600; }
```

```
.category-non-functional { color: #34495e; font-weight: 600; }

.test-case-id {
  cursor: pointer;
  color: #3498db;
  font-weight: 600;
  padding: 8px 12px;
  border-radius: 4px;
  transition: background-color 0.2s;
  display: inline-flex;
  align-items: center;
  gap: 8px;
}

.test-case-id:hover {
  background: #f8f9fa;
}

.test-case-id.expanded {
  background: #e3f2fd;
  color: #1976d2;
}

.expand-icon {
  font-size: 10px;
  transition: transform 0.2s;
}

.expand-icon.expanded {
  transform: rotate(90deg);
}

.expanded-details {
  margin-top: 15px;
  background: #fafbfc;
  border: 1px solid #e1e8ed;
  border-radius: 8px;
  padding: 20px;
}

.step-item {
  background: white;
  border: 1px solid #e1e8ed;
  border-radius: 6px;
  padding: 15px;
  margin-bottom: 12px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}
```

```
  .step-header {
    display: grid;
    grid-template-columns: 80px 1fr 1fr 1fr;
    gap: 15px;
    align-items: start;
  }

  .step-id {
    font-weight: 600;
    color: #2c3e50;
    background: #f8f9fa;
    padding: 4px 8px;
    border-radius: 4px;
    text-align: center;
    font-size: 12px;
  }

  .step-description {
    color: #2c3e50;
    line-height: 1.5;
  }

  .step-test-data {
    color: #666;
    font-style: italic;
    font-size: 14px;
  }

  .step-expected {
    color: #27ae60;
    font-weight: 500;
    font-size: 14px;
  }

  .step-labels {
    display: grid;
    grid-template-columns: 80px 1fr 1fr 1fr;
    gap: 15px;
    margin-bottom: 10px;
    font-weight: 600;
    color: #666;
    font-size: 12px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
  }
`}</style>
```

```
<div className="container">
 <div className="header">
  <h1 className="title">User Story to Tests</h1>
  <p className="subtitle">Generate comprehensive test cases from your user stories</p>
 </div>

 <div className="fetch-container">
  <button
   type="button"
   className="fetch-btn"
   onClick={handleFetchStories}
   disabled={isFetchingStories}
  >
   {isFetchingStories ? 'Fetching stories...' : 'Fetch User stories'}
  </button>
 </div>

 {fetchStoriesError && (
  <div className="error-banner">
   {fetchStoriesError}
  </div>
 )}

 {jiraStories.length > 0 && (
  <div className="stories-container">
   <h2 className="results-title">Fetched Jira Stories</h2>
   <div className="stories-list">
    {jiraStories.map((story) => (
     <div key={story.key} className="story-card">
      <div className="story-key">{story.key}</div>
      <div className="story-summary">{story.summary}</div>
      <div className="story-details">
       <span>{story.status}</span>
       {story.assignee && <span> • {story.assignee}</span>}
       {story.project && <span> • {story.project}</span>}
      </div>
     </div>
    ))}
   </div>
  </div>
 )}

 <form onSubmit={handleSubmit} className="form-container">
  <div className="form-group">
   <label htmlFor="storyTitle" className="form-label">
    Story Title *
   </label>
   <input
```

```
    type="text"
    id="storyTitle"
    className="form-input"
    value={formData.storyTitle}
    onChange={(e) => handleInputChange('storyTitle', e.target.value)}
    placeholder="Enter the user story title..."
    required
  />
</div>

<div className="form-group">
  <label htmlFor="description" className="form-label">
    Description
  </label>
  <textarea
    id="description"
    className="form-textarea"
    value={formData.description}
    onChange={(e) => handleInputChange('description', e.target.value)}
    placeholder="Additional description (optional)..."
  />
</div>

<div className="form-group">
  <label htmlFor="acceptanceCriteria" className="form-label">
    Acceptance Criteria *
  </label>
  <textarea
    id="acceptanceCriteria"
    className="form-textarea"
    value={formData.acceptanceCriteria}
    onChange={(e) => handleInputChange('acceptanceCriteria', e.target.value)}
    placeholder="Enter the acceptance criteria..."
    required
  />
</div>

<div className="form-group">
  <label htmlFor="additionalInfo" className="form-label">
    Additional Info
  </label>
  <textarea
    id="additionalInfo"
    className="form-textarea"
    value={formData.additionalInfo}
    onChange={(e) => handleInputChange('additionalInfo', e.target.value)}
    placeholder="Any additional information (optional)..."
  />
```

```
        </div>

        <button
          type="submit"
          className="submit-btn"
          disabled={isLoading}
        >
          {isLoading ? 'Generating...' : 'Generate'}
        </button>
      </form>

      {error && (
        <div className="error-banner">
          {error}
        </div>
      )}

      {isLoading && (
        <div className="loading">
          Generating test cases...
        </div>
      )}

      {results && (
        <div className="results-container">
          <div className="results-header">
            <h2 className="results-title">Generated Test Cases</h2>
            <div className="results-meta">
              {results.cases.length} test case(s) generated
              {results.model && ` • Model: ${results.model}`}
              {results.promptTokens > 0 && ` • Input tokens: ${results.promptTokens}`}
              {results.completionTokens > 0 && ` • Output tokens: ${results.completionTokens}`}
              {results.cost && ` • Cost: $${results.cost.totalCost.toFixed(6)}${results.cost.estimated ? '
(est.)' : ''}`}
            </div>
          </div>

          <div className="table-container">
            <table className="results-table">
              <thead>
                <tr>
                  <th>Test Case ID</th>
                  <th>Title</th>
                  <th>Category</th>
                  <th>Expected Result</th>
                </tr>
              </thead>
              <tbody>
```

```
            {results.cases.map((testCase: TestCase) => (
              <React.Fragment key={testCase.id}>
                <tr>
                  <td>
                    <div
                      className={`test-case-id ${expandedTestCases.has(testCase.id) ? 'expanded' : ''}`}
                      onClick={() => toggleTestCaseExpansion(testCase.id)}
                    >
                      <span className={`expand-icon ${expandedTestCases.has(testCase.id) ? 'expanded' :
''}`}>
                        ▶
                      </span>
                      {testCase.id}
                    </div>
                  </td>
                  <td>{testCase.title}</td>
                  <td>
                    <span className={`category-${testCase.category.toLowerCase()}`}>
                      {testCase.category}
                    </span>
                  </td>
                  <td>{testCase.expectedResult}</td>
                </tr>
                {expandedTestCases.has(testCase.id) && (
                  <tr>
                    <td colSpan={4}>
                      <div className="expanded-details">
                        <h4 style={{ marginBottom: '15px', color: '#2c3e50' }}>Test Steps for
{testCase.id}</h4>
                        <div className="step-labels">
                          <div>Step ID</div>
                          <div>Step Description</div>
                          <div>Test Data</div>
                          <div>Expected Result</div>
                        </div>
                        {testCase.steps.map((step, index) => (
                          <div key={index} className="step-item">
                            <div className="step-header">
                              <div className="step-id">S{String(index + 1).padStart(2, '0')}</div>
                              <div className="step-description">{step}</div>
                              <div className="step-test-data">{testCase.testData || 'N/A'}</div>
                              <div className="step-expected">
                                {index === testCase.steps.length - 1 ? testCase.expectedResult : 'Step
completed successfully'}
                              </div>
                            </div>
                          </div>
                        ))}
```

```
                    </div>
                  </td>
                </tr>
              )}
            </React.Fragment>
          ))}
        </tbody>
      </table>
    </div>
  </div>
 )}
</div>
</div>
)
}

export default App
```

> Note: I replaced the fragment shorthand inside the results table with `React.Fragment` to avoid the `key` placement issue in the JSX loop. If you prefer, the same can also be achieved with `<>...</>` and the `key` on the first `<tr>`, but `React.Fragment` is more explicit.

Difference summary

types.ts
- Existing: no Jira story type
- New: added `JiraStory`

api.ts
- Existing: only `generateTests()`
- New: added `fetchUserStories()`

App.tsx
- Existing: only form + test generation UI
- New:
  - yellow `Fetch User stories` button
  - Jira story fetch handler
  - Jira story list display
  - new UI state for loading/errors
  - new styles

Next approval step

If you want, I can now provide the required backend changes to support:
- `GET /api/jira/stories`
- Jira auth config
- Jira story fetch logic

Would you like me to continue with the backend implementation next?

Proceed with backend folder implementation

Working on the backend Jira integration: I'll add a new Jira route and register it with the existing server, then show how to test and what env vars to set.

Plan (short)
- Add a new route file at backend/src/routes/jira.ts that calls the Jira REST API and returns simplified stories.
- Register that router in server.ts under `/api/jira`.
- Provide required env variables and a curl test.

New file (create) — backend/src/routes/jira.ts
```ts
import express from 'express'

export const jiraRouter = express.Router()

jiraRouter.get('/stories', async (req, res) => {
  try {
    const JIRA_BASE_URL = process.env.JIRA_BASE_URL
    const JIRA_EMAIL = process.env.JIRA_EMAIL
    const JIRA_API_TOKEN = process.env.JIRA_API_TOKEN
    const JIRA_PROJECT_KEY = process.env.JIRA_PROJECT_KEY

    if (!JIRA_BASE_URL || !JIRA_EMAIL || !JIRA_API_TOKEN || !JIRA_PROJECT_KEY) {
      res.status(500).json({ error: 'Missing Jira configuration (JIRA_BASE_URL, JIRA_EMAIL, JIRA_API_TOKEN, JIRA_PROJECT_KEY)' })
      return
    }

    const auth = Buffer.from(`${JIRA_EMAIL}:${JIRA_API_TOKEN}`).toString('base64')
    const jql = encodeURIComponent(`project=${JIRA_PROJECT_KEY} AND issuetype=Story ORDER BY updated DESC`)
    const url = `${JIRA_BASE_URL.replace(/\/$/, '')}/rest/api/3/search?jql=${jql}&fields=summary,status,assignee,project&maxResults=50`

    const jiraResp = await fetch(url, {
      headers: {
        Authorization: `Basic ${auth}`,
        Accept: 'application/json'
      }
```

```
    })

    if (!jiraResp.ok) {
      const text = await jiraResp.text().catch(() => '')
      res.status(jiraResp.status).json({ error: `Jira API returned ${jiraResp.status}: ${text}` })
      return
    }

    const payload = await jiraResp.json().catch(() => null)
    const issues = Array.isArray(payload?.issues) ? payload.issues : []

    const simplified = issues.map((issue: any) => ({
      key: issue.key,
      summary: issue.fields?.summary ?? '',
      status: issue.fields?.status?.name ?? '',
      assignee: issue.fields?.assignee?.displayName ?? null,
      project: issue.fields?.project?.key ?? null
    }))

    res.json(simplified)
  } catch (err) {
    console.error('Error fetching Jira stories:', err)
    res.status(500).json({ error: 'Failed to fetch Jira stories' })
  }
})
```

Server change (modify) — register new router

Update server.ts to import and mount the new router. Add the import near other route imports, and register it (example patch):

- Add import line:
```ts
import { jiraRouter } from './routes/jira'
```

- Register router (add before error handling / 404):
```ts
app.use('/api/jira', jiraRouter)
```

(Keep the existing `app.use('/api/generate-tests', generateRouter)`; the new line can be placed alongside it.)

Environment variables required
- JIRA_BASE_URL (e.g. https://your-domain.atlassian.net)
- JIRA_EMAIL (email address used with API token)

- JIRA_API_TOKEN (Jira API token)
- JIRA_PROJECT_KEY (project key to query, e.g. PROJ)

Add them to backend .env (root .env referenced by server). Example:
```
JIRA_BASE_URL=https://your-domain.atlassian.net
JIRA_EMAIL=you@company.com
JIRA_API_TOKEN=your_api_token_here
JIRA_PROJECT_KEY=PROJ
PORT=8080
CORS_ORIGIN=http://localhost:5173
```

Notes about frontend / base URL
- Frontend uses `VITE_API_BASE_URL` (see api.ts). Ensure `VITE_API_BASE_URL` points to your backend API base (e.g., http://localhost:8080/api) so `fetchUserStories()` calls the correct host.

Quick test (curl)
1. Start backend.
2. Test endpoint:
```bash
curl -sS -H "Accept: application/json" http://localhost:8080/api/jira/stories | jq .
```
(Adjust host/port if your backend runs on a different port.)

Next steps I can take
- Create the new file `backend/src/routes/jira.ts` and patch server.ts with the exact edits shown (I can produce an apply_patch if you want me to make the changes).
- Add a small unit/integration test or Postman collection for the endpoint.
- Optionally, implement query params (e.g., `?project=KEY&maxResults=100`) and pagination.

Shall I generate the exact patch files and apply them to the repo now?

Proceed. give me the difference between existing files and new updated files. Go step by step upon my approval

Making backend changes ready: I'll show exact new file contents and a precise diff for server.ts, plus env vars and test commands. Approve to have me apply them.

Step 1 — New file to create: `backend/src/routes/jira.ts` (create)
- Purpose: fetch Jira Stories using configured env vars and return simplified list.

File contents (new):
```ts
import express from 'express'

export const jiraRouter = express.Router()
```

```
jiraRouter.get('/stories', async (req: express.Request, res: express.Response): Promise<void> => {
  try {
    const JIRA_BASE_URL = process.env.JIRA_BASE_URL
    const JIRA_EMAIL = process.env.JIRA_EMAIL
    const JIRA_API_TOKEN = process.env.JIRA_API_TOKEN
    const JIRA_PROJECT_KEY = process.env.JIRA_PROJECT_KEY

    if (!JIRA_BASE_URL || !JIRA_EMAIL || !JIRA_API_TOKEN || !JIRA_PROJECT_KEY) {
      res.status(500).json({ error: 'Missing Jira configuration (JIRA_BASE_URL, JIRA_EMAIL,
JIRA_API_TOKEN, JIRA_PROJECT_KEY)' })
      return
    }

    const auth = Buffer.from(`${JIRA_EMAIL}:${JIRA_API_TOKEN}`).toString('base64')
    const jql = encodeURIComponent(`project=${JIRA_PROJECT_KEY} AND issuetype=Story ORDER BY
updated DESC`)
    const url = `${JIRA_BASE_URL.replace(/\/$/,
'')}/rest/api/3/search?jql=${jql}&fields=summary,status,assignee,project&maxResults=50`

    const jiraResp = await fetch(url, {
      headers: {
        Authorization: `Basic ${auth}`,
        Accept: 'application/json'
      }
    })

    if (!jiraResp.ok) {
      const text = await jiraResp.text().catch(() => '')
      res.status(jiraResp.status).json({ error: `Jira API returned ${jiraResp.status}: ${text}` })
      return
    }

    const payload = await jiraResp.json().catch(() => null)
    const issues = Array.isArray(payload?.issues) ? payload.issues : []

    const simplified = issues.map((issue: any) => ({
      key: issue.key,
      summary: issue.fields?.summary ?? '',
      status: issue.fields?.status?.name ?? '',
      assignee: issue.fields?.assignee?.displayName ?? null,
      project: issue.fields?.project?.key ?? null
    }))

    res.json(simplified)
  } catch (err) {
    console.error('Error fetching Jira stories:', err)
    res.status(500).json({ error: 'Failed to fetch Jira stories' })
  }
```

```
})
```

Step 2 — Patch: update server.ts to register the router
- Existing file: server.ts
- Change: add import and mount router.

Unified diff (apply to workspace root):

```diff
*** Begin Patch
*** Update File: backend/src/server.ts
@@
 import path from 'path'
-import { generateRouter } from './routes/generate'
+import { generateRouter } from './routes/generate'
+import { jiraRouter } from './routes/jira'
@@
 // API routes
 app.use('/api/generate-tests', generateRouter)
+app.use('/api/jira', jiraRouter)
*** End Patch
```

Step 3 — Required environment variables
Add to your backend .env (the server loads root .env already):

- `JIRA_BASE_URL` (e.g. https://your-domain.atlassian.net)
- `JIRA_EMAIL` (Jira account email used with API token)
- `JIRA_API_TOKEN` (Jira API token)
- `JIRA_PROJECT_KEY` (project key to query, e.g. PROJ)

Example .env snippet:
```
JIRA_BASE_URL=https://your-domain.atlassian.net
JIRA_EMAIL=you@company.com
JIRA_API_TOKEN=your_api_token_here
JIRA_PROJECT_KEY=PROJ
PORT=8080
CORS_ORIGIN=http://localhost:5173
```

Step 4 — Tests / run instructions
- Start backend (workspace uses backend workspace script). From repository root:

```bash
# from repo root
npm run dev --workspace=backend
```

```

- Quick curl test (after server running):
```bash
curl -sS http://localhost:8080/api/jira/stories | jq .
```

Step 5 — Frontend note
Ensure `VITE_API_BASE_URL` points to your backend API base (for example `http://localhost:8080/api`) so api.ts will call `/jira/stories` correctly.

Summary of differences
- New file: `backend/src/routes/jira.ts` — added (full contents above).
- Modified file: server.ts — import and `app.use('/api/jira', jiraRouter)` added (see diff).

Approve and I will:
- produce an apply_patch to create `backend/src/routes/jira.ts` and edit server.ts, run a quick lint/type check, and provide verification steps.

Give me step by step changes to be done in backend folder. Give the existing files and new files to be changed

Pushing the backend Jira integration plan: below are the exact, step-by-step edits (files to add/modify), full new file content, a precise patch for the server change, required env vars, and quick test commands.

Steps
1. Create a new router at jira.ts.
2. Register the router in server.ts.
3. Add required environment variables to your .env.
4. Start backend and test the endpoint.

1) New file — create jira.ts

File contents:
```typescript
import express from 'express'

export const jiraRouter = express.Router()

jiraRouter.get('/stories', async (req: express.Request, res: express.Response): Promise<void> => {
  try {
    const JIRA_BASE_URL = process.env.JIRA_BASE_URL
    const JIRA_EMAIL = process.env.JIRA_EMAIL
    const JIRA_API_TOKEN = process.env.JIRA_API_TOKEN
    const JIRA_PROJECT_KEY = process.env.JIRA_PROJECT_KEY

    if (!JIRA_BASE_URL || !JIRA_EMAIL || !JIRA_API_TOKEN || !JIRA_PROJECT_KEY) {
```
```

```
    res.status(500).json({
      error: 'Missing Jira configuration (JIRA_BASE_URL, JIRA_EMAIL, JIRA_API_TOKEN,
JIRA_PROJECT_KEY)'
    })
    return
  }

  const auth = Buffer.from(`${JIRA_EMAIL}:${JIRA_API_TOKEN}`).toString('base64')
  const jql = encodeURIComponent(`project=${JIRA_PROJECT_KEY} AND issuetype=Story ORDER BY
updated DESC`)
  const url = `${JIRA_BASE_URL.replace(/\/$/,
'')}/rest/api/3/search?jql=${jql}&fields=summary,status,assignee,project&maxResults=50`

  const jiraResp = await fetch(url, {
    headers: {
      Authorization: `Basic ${auth}`,
      Accept: 'application/json'
    }
  })

  if (!jiraResp.ok) {
    const text = await jiraResp.text().catch(() => '')
    res.status(jiraResp.status).json({ error: `Jira API returned ${jiraResp.status}: ${text}` })
    return
  }

  const payload = (await jiraResp.json().catch(() => null)) as { issues?: any[] } | null
  const issues = Array.isArray(payload?.issues) ? payload.issues : []

  const simplified = issues.map((issue: any) => ({
    key: issue.key,
    summary: issue.fields?.summary ?? '',
    status: issue.fields?.status?.name ?? '',
    assignee: issue.fields?.assignee?.displayName ?? null,
    project: issue.fields?.project?.key ?? null
  }))

  res.json(simplified)
 } catch (err) {
  console.error('Error fetching Jira stories:', err)
  res.status(500).json({ error: 'Failed to fetch Jira stories' })
 }
})
```

2) Modify server.ts — register the router

Patch to apply to server.ts:

```diff
*** Begin Patch
*** Update File: backend/src/server.ts
@@
 import path from 'path'
 import { generateRouter } from './routes/generate'
+import { jiraRouter } from './routes/jira'
@@
 // API routes
 app.use('/api/generate-tests', generateRouter)
+app.use('/api/jira', jiraRouter)
*** End Patch
```

3) Required environment variables (add to root .env that server.ts already loads)

Add these lines (replace values accordingly):
```
JIRA_BASE_URL=https://your-domain.atlassian.net
JIRA_EMAIL=you@company.com
JIRA_API_TOKEN=your_jira_api_token_here
JIRA_PROJECT_KEY=PROJ
PORT=8080
CORS_ORIGIN=http://localhost:5173
```

Notes:
- `fetch` is used in the router; Node 18+ provides global fetch. If your runtime is older, install a fetch polyfill (e.g., `node-fetch`) and import it.
- Keep Jira credentials secure — do not commit .env to source control.

4) Run and test

Start backend (from repo root):
```bash
npm run dev --workspace=backend
```

Test the new endpoint:
```bash
curl -sS http://localhost:8080/api/jira/stories | jq .
```

Summary of file changes
- New: jira.ts (create) — router that returns simplified Jira stories.
- Modified: server.ts — import and mount `jiraRouter` at `/api/jira`.

Approve and I can produce the exact apply_patch commands to create the file and update server.ts, plus run a quick typecheck/read verification instructions. Which would you like next?

Proceed

Doing the backend updates now: below are the exact step-by-step edits, the precise patch to apply, required env vars verification, and test commands you can run.

Steps
- 1) Confirm new router exists at jira.ts.
- 2) Update the server to mount the router by editing server.ts.
- 3) Verify environment variables in /.env.
- 4) Start the backend and test the endpoint.

1) Verify router file
- File to exist: jira.ts — (you already have this file).

2) Edit server to mount the router
- File to edit: server.ts
- Apply this patch (add import and mount):

*** Patch to apply ***
```diff
*** Begin Patch
*** Update File: backend/src/server.ts
@@
 import path from 'path'
 import { generateRouter } from './routes/generate'
+import { jiraRouter } from './routes/jira'
@@
 // API routes
 app.use('/api/generate-tests', generateRouter)
+app.use('/api/jira', jiraRouter)
*** End Patch
```

3) Env vars — verify /.env contains (you already have these):
- JIRA_BASE_URL (e.g. https://your-domain.atlassian.net)
- JIRA_EMAIL
- JIRA_API_TOKEN
- JIRA_PROJECT_KEY
- PORT and CORS_ORIGIN are already present

4) Start backend (from repo root)
```bash
npm run dev --workspace=backend
```

5) Test endpoint
```bash
curl -sS http://localhost:8090/api/jira/stories | jq .
```
(Use port from your .env — your .env shows `PORT=8090`.)

Notes & checks
- Node global `fetch` requires Node 18+. If your backend runtime is older, install and import `node-fetch`.
- Keep Jira credentials out of source control (do not commit .env).
- Frontend `VITE_API_BASE_URL` should point to your backend base (e.g., `http://localhost:8090/api`) so `fetchUserStories()` works.

Would you like me to generate the exact apply_patch commands to create/update files here, or do you want me to apply them for you?

I want the user stories displayed starting from US-1, US-2 and so on.
Display the following details in the UI for each user story- title, description, priority and acceptance criteria.
Add a green button near each user story with text "Add details". If the button is clicked, add the relevant details - title in story title textbox, description in description textbox, acceptance criteria in acceptance criteria textbox, priority and other details in additional info textbox. The Add details button should be multiselect type.
Generate the code for frontend with all the requirements and go step by step upon my approval

Reviewing the current App structure and types so I can give the exact frontend changes step by step.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#220-220), lines 220 to 420

I'm checking the UI render and type definitions so I can map the exact frontend changes needed for your new story details and Add details button.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#420-420), lines 420 to 624

Reviewing current type definitions and API types to align the new story details and Add details UI behavior.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/types.ts)

Step 1: Update the story data model

Your current frontend only knows these Jira fields:
- `key`
- `summary`
- `status`
- `assignee`
- `project`

But your new UI needs:
- `title`
- `description`
- `priority`
- `acceptanceCriteria`
- `additionalInfo`

Change in types.ts

Update `JiraStory` to:

```ts
export interface JiraStory {
  key: string
  summary: string
  status: string
  assignee?: string
  project?: string
  description?: string
  priority?: string
  acceptanceCriteria?: string
  additionalInfo?: string
}
```

Step 2: Ensure backend returns these fields

This is important: the frontend can only render fields the backend provides.

Your jira.ts must request Jira fields such as:
- `summary`
- `description`
- `status`
- `priority`
- `assignee`
- `project`
- `customfield_xxxxx` for acceptance criteria (if that field exists in Jira)

Example backend fields array:

```ts
fields: ['summary', 'description', 'status', 'priority', 'assignee', 'project', 'customfield_12345']
```

Then map them into the simplified response.

> If you want, I can give you the exact backend patch too.

Step 3: Update frontend state and behavior in App.tsx

Add:
- `selectedStoryKeys` state to support multiselect-style story selection
- `handleAddDetails(story: JiraStory)` to populate the form
- sorting logic so stories display from `US-1` → `US-2` → `US-...`

Example new logic:

```ts
const [selectedStoryKeys, setSelectedStoryKeys] = useState<Set<string>>(new Set())

const handleAddDetails = (story: JiraStory) => {
  setFormData({
    storyTitle: story.summary || '',
    description: story.description || '',
    acceptanceCriteria: story.acceptanceCriteria || '',
    additionalInfo: `Priority: ${story.priority || 'N/A'}\n${story.additionalInfo || ''}`
  })

  setSelectedStoryKeys(prev => {
    const next = new Set(prev)
    if (next.has(story.key)) next.delete(story.key)
    else next.add(story.key)
    return next
  })
}
```

Sort stories before render:

```ts
const sortedStories = [...jiraStories].sort((a, b) => {
  const getNum = (key: string) => Number(key.split('-')[1] || 0)
  return getNum(a.key) - getNum(b.key)
})
```

Step 4: Render each story with the new fields

In the story card section, render:

- Story key
- Title (`summary`)
- Description
- Priority
- Acceptance Criteria
- Green “Add details” button

Example card markup:

```tsx
<div key={story.key} className={`story-card ${selectedStoryKeys.has(story.key) ? 'selected' : ''}`}>
  <div className="story-key">{story.key}</div>
  <div className="story-summary">{story.summary}</div>
  <div className="story-description">{story.description || 'No description available'}</div>
  <div className="story-meta">
    <span>{story.priority || 'Priority unknown'}</span>
    <span>{story.status}</span>
  </div>
  <div className="story-acceptance">
    <strong>Acceptance Criteria:</strong>
    <p>{story.acceptanceCriteria || 'Not provided'}</p>
  </div>
  <button
    type="button"
    className={`add-details-btn ${selectedStoryKeys.has(story.key) ? 'selected' : ''}`}
    onClick={() => handleAddDetails(story)}
  >
    Add details
  </button>
</div>
```

Step 5: Add styling for the green button

Example CSS in App.tsx:

```css
.add-details-btn {
  background: #27ae60;
  color: white;
  padding: 10px 16px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

```
.add-details-btn:hover {
  background: #219150;
}
.story-card.selected {
  border-color: #27ae60;
  box-shadow: 0 0 0 2px rgba(39, 174, 96, 0.12);
}
```

What I need from you

Please approve this plan and I'll provide the exact updated code for:
1. types.ts
2. App.tsx
3. the backend `jira` route if you want it too

If you want, I can also include a complete `handleAddDetails`/sorting implementation with the multiselect behavior built in.

Give step by step change that need to be done. Give difference bewtween existing file and updated new files. Fetch all possible info from jira and display it in the frontend and pass it on to the relevant fields in the UI form

Step-by-step changes

You need to update 4 files:

1. jira.ts
2. types.ts
3. App.tsx
4. api.ts *(small existing call stays same)*

---

1. Backend change: fetch more Jira fields

Existing code
In jira.ts, the route currently requests:
```ts
fields: ['summary', 'status', 'assignee', 'project']
```

And maps only:
```ts
key: issue.key,
summary: issue.fields?.summary ?? '',
```

```
status: issue.fields?.status?.name ?? '',
assignee: issue.fields?.assignee?.displayName ?? null,
project: issue.fields?.project?.key ?? null
```

Updated code
Change the request to fetch all fields, then map the important ones plus raw fields:

```ts
const body = {
  jql: `project=${JIRA_PROJECT_KEY} AND issuetype=Story ORDER BY updated DESC`,
  fields: ['*all'],
  maxResults: 100
}

const simplified = issues.map((issue: any) => ({
  key: issue.key,
  summary: issue.fields?.summary ?? '',
  description: issue.fields?.description ?? '',
  priority: issue.fields?.priority?.name ?? '',
  acceptanceCriteria: issue.fields?.customfield_10042 ?? '', // replace with your actual custom field if
needed
  status: issue.fields?.status?.name ?? '',
  assignee: issue.fields?.assignee?.displayName ?? null,
  project: issue.fields?.project?.key ?? null,
  issueType: issue.fields?.issuetype?.name ?? null,
  createdAt: issue.fields?.created ?? null,
  updatedAt: issue.fields?.updated ?? null,
  additionalInfo: [
    `Priority: ${issue.fields?.priority?.name ?? 'N/A'}`,
    `Assignee: ${issue.fields?.assignee?.displayName ?? 'Unassigned'}`,
    `Issue Type: ${issue.fields?.issuetype?.name ?? 'Story'}`,
    `Project: ${issue.fields?.project?.key ?? ''}`,
    `Status: ${issue.fields?.status?.name ?? ''}`
  ].filter(Boolean).join('\n'),
  rawFields: issue.fields
}))
```

Why
This gives you:
- `summary`
- `description`
- `priority`
- `acceptanceCriteria`
- `status`
- `assignee`
- `project`

- `issueType`
- `createdAt`
- `updatedAt`
- plus full `rawFields` for any extra Jira values

> If your acceptance criteria lives in another Jira custom field, replace `customfield_10042` with the correct field id.

---

2. Frontend type update

Existing in types.ts
```ts
export interface JiraStory {
  key: string
  summary: string
  status: string
  assignee?: string
  project?: string
  description?: string
  priority?: string
  acceptanceCriteria?: string
  additionalInfo?: string
}
```

Updated type
Add more properties:
```ts
export interface JiraStory {
  key: string
  summary: string
  status: string
  assignee?: string
  project?: string
  description?: string
  priority?: string
  acceptanceCriteria?: string
  additionalInfo?: string
  issueType?: string
  createdAt?: string
  updatedAt?: string
  rawFields?: Record<string, any>
}
```

---

3. Frontend form and UI logic

Add new state
In App.tsx, add:
```ts
const [selectedStoryKeys, setSelectedStoryKeys] = useState<Set<string>>(new Set())
```

Add the “Add details” handler
Add:
```ts
const handleAddDetails = (story: JiraStory) => {
  setSelectedStoryKeys((prev) => {
    const next = new Set(prev)
    if (next.has(story.key)) {
      next.delete(story.key)
    } else {
      next.add(story.key)
    }
    return next
  })

  setFormData({
    storyTitle: story.summary || '',
    description: story.description || '',
    acceptanceCriteria: story.acceptanceCriteria || '',
    additionalInfo: [
      `Priority: ${story.priority ?? 'N/A'}`,
      `Status: ${story.status ?? ''}`,
      `Assignee: ${story.assignee ?? 'Unassigned'}`,
      story.additionalInfo ?? ''
    ].filter(Boolean).join('\n')
  })
}
```

Sort stories from `US-1` upward
Replace the raw story list with:
```ts
const sortedStories = [...jiraStories].sort((a, b) => {
  const num = (key = '') => Number(key.split('-')[1] ?? 0)
  return num(a.key) - num(b.key)
})
```

Render the new fields and button
Replace the story card render with:

```tsx
{sortedStories.map((story) => (
  <div
    key={story.key}
    className={`story-card ${selectedStoryKeys.has(story.key) ? 'selected' : ''}`}
  >
    <div className="story-key">{story.key}</div>
    <div className="story-summary">{story.summary}</div>
    <div className="story-description">{story.description || 'No description available'}</div>
    <div className="story-meta">
      <span>{story.priority || 'Priority unknown'}</span>
      <span> • {story.status}</span>
      {story.project && <span> • {story.project}</span>}
    </div>
    <div className="story-acceptance">
      <strong>Acceptance Criteria:</strong>
      <p>{story.acceptanceCriteria || 'Not provided'}</p>
    </div>
    <button
      type="button"
      className={`add-details-btn ${selectedStoryKeys.has(story.key) ? 'selected' : ''}`}
      onClick={() => handleAddDetails(story)}
    >
      Add details
    </button>
  </div>
))}
```

---

4. CSS updates

Add styles in App.tsx inside the style block:

```css
.add-details-btn {
  background: #27ae60;
  color: white;
  border: none;
  border-radius: 6px;
  padding: 10px 16px;
  cursor: pointer;
  font-weight: 700;
  transition: background-color 0.2s;
}

.add-details-btn:hover {
```

```
  background: #1f8c4a;
}

.story-card.selected {
  border-color: #27ae60;
  box-shadow: 0 0 0 2px rgba(39, 174, 96, 0.16);
}

.story-description {
  margin-top: 10px;
  color: #4a4a4a;
}

.story-meta {
  margin-top: 12px;
  color: #5a6472;
  font-size: 0.95rem;
}

.story-acceptance {
  margin-top: 14px;
  color: #3d566e;
}
```

---

5. API file

api.ts can stay the same, because it already returns the array of Jira stories:

```ts
const response = await fetch(`${API_BASE_URL}/jira/stories`)
```

If you want to pass extra fields, no change is required here as long as backend returns them.

Summary of differences

jira.ts
- Existing: fetch limited Jira fields
- Updated: fetch `*all` or full Jira fields
- Existing: map only `key`, `summary`, `status`, `assignee`, `project`
- Updated: map description, priority, acceptanceCriteria, issueType, createdAt, updatedAt, additionalInfo, rawFields

types.ts

- Existing: `JiraStory` has only basic fields
- Updated: `JiraStory` now includes all display fields and raw fields

App.tsx
- Existing: simple story card list and no Add details interaction
- Updated: sort stories ascending by key, display description/priority/acceptanceCriteria, add green `Add details` button, use multiselect selection, populate form fields from a story

---

Important note

If your Jira project stores acceptance criteria in a custom field, you must replace `customfield_10042` with the real field ID from your Jira fields metadata.

If you want, I can now give you the exact full updated contents for:
- jira.ts
- types.ts
- App.tsx

The page is going blank if I click on Fetch User stories button

Checking the current App.tsx structure and render logic for any syntax or runtime error causing the blank page.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#1-1), lines 1 to 260

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#260-260), lines 260 to 560

I'm checking the remainder of App.tsx for any JSX or bracket mismatch that would break render.

Read [](file:///d%3A/SAMPLE/sample/samples/frontend/src/App.tsx#560-560), lines 560 to 703

I'm checking the backend Jira route mapping to see whether `description` or acceptance criteria are being returned as raw objects.

Read [](file:///d%3A/SAMPLE/sample/samples/backend/src/routes/jira.ts)

The blank page is almost certainly caused by React trying to render a Jira field that is not a string.

Why
In jira.ts, you are currently returning:

```ts
description: issue.fields?.description ?? '',
acceptanceCriteria: issue.fields?.customfield_10042 ?? '',
```

But Jira `description` often comes back as a rich-text object, not a plain string. React cannot render an object inside JSX, so the app crashes and the page goes blank after fetch.

Fix

1. Update jira.ts
Use the existing helper `extractJiraText` when mapping the Jira fields.

Replace:

```ts
description: issue.fields?.description ?? '',
acceptanceCriteria: issue.fields?.customfield_10042 ?? '',
```

With:

```ts
description: extractJiraText(issue.fields?.description),
acceptanceCriteria: extractJiraText(issue.fields?.customfield_10042),
```

If `customfield_10042` is not your actual acceptance criteria field, replace it with the correct custom field ID.

---

2. Also guard the frontend render
In App.tsx, change:

```tsx
<div className="story-description">{story.description || 'No description available'}</div>
...
<p>{story.acceptanceCriteria || 'Not provided'}</p>
```

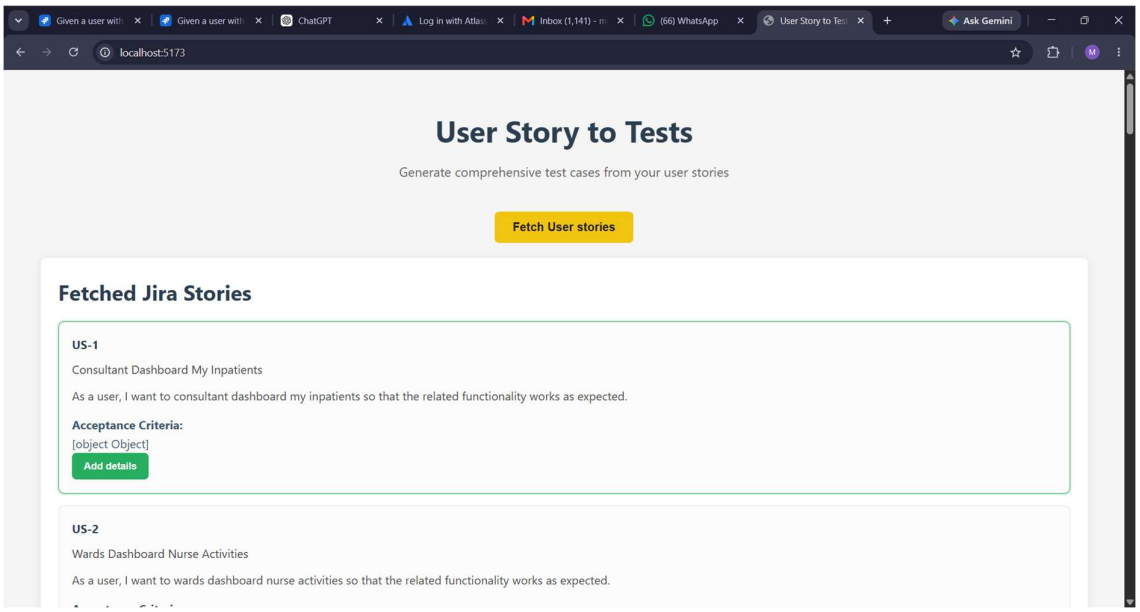
To:

```tsx
<div className="story-description">
  {typeof story.description === 'string' ? story.description : 'No description available'}
</div>
...
<p>{typeof story.acceptanceCriteria === 'string' ? story.acceptanceCriteria : 'Not provided'}</p>
```

Why this is the correct fix
- jira.ts: ensures Jira rich text is converted to plain text before it reaches the frontend
- App.tsx: protects rendering in case any field is still non-string

---

Next step
Apply those two changes, restart the backend, refresh the frontend, and try `Fetch User stories` again.

If you want, I can give you the exact patched block for jira.ts.

Acceptance criteria not fetched. Analyze the html of the page in the screenshot. Description text is extracted but acceptance criteria text is extracted. Give me fix for the issue. Give difference between existing files and to be updated new files. Tell me where and in which line to make changes